Soumiya Ayyappan_W6_D4

Code and its output:

Step 1:

Code:

codecod3.py

day1 > codecod3.py > load_feedback_data

```python
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import col, trim, min, max, avg

#1. Load Feedback Data

def load_feedback_data(spark: SparkSession, path: str) -> DataFrame:
    return spark.read.csv(
    path,
    header=True,
    inferSchema=True
    )
```

Output:

```
+-----------------+------+---------+
|            email|rating|  comment|
+-----------------+------+---------+
|  alice@gmail.com|     5|Excellent|
|                 |     4|     Good|
|             NULL|     3|  Average|
|    bob@gmail.com|     4|     Good|
|charlie@gmail.com|     3|  Average|
|  david@gmail.com|     5|Excellent|
+-----------------+------+---------+
```

Step 2:

Code:

```python
#2. Clean Missing and InvalidEmails

def clean_missing_and_invalid_emails(df: DataFrame) -> DataFrame:
    df = df.filter(
    col("email").isNotNull() &
    (trim(col("email")) != "")
    )
    return df.select("email")
```

Output:

```
2. Cleaned Emails:
+-----------------+
|            email|
+-----------------+
|  alice@gmail.com|
|    bob@gmail.com|
|charlie@gmail.com|
|  david@gmail.com|
+-----------------+
```

Step 3:

Code:

```python
#3. Compute Rating Statistics

def compute_rating_stats(df: DataFrame) -> dict:
    df = df.withColumn(
    "rating",
    col("rating").cast("int")
    )

    df = df.filter(
        col("rating").isNotNull()
    )

    result = df.agg(
        min("rating").alias("min_rating"),
        max("rating").alias("max_rating"),
        avg("rating").alias("avg_rating")
    ).collect()[0]

    return {
        "min_rating": result["min_rating"],
        "max_rating": result["max_rating"],
        "avg_rating": result["avg_rating"]
    }

```

Output:

```
3. Rating Statistics:
{'min_rating': 3, 'max_rating': 5, 'avg_rating': 4.0}
```

Step 4:

Code:

```
46  #4. Count Ratings
47
48  def count_ratings(df: DataFrame) -> int:
49      return df.filter(
50      col("rating").isNotNull()
51      ).count()
```

Output:

```
4. Number of Ratings:
6
```

Step 5:

Code:

```python
#5. Most Common Rating

def most_common_rating(df: DataFrame) -> int:
    df = df.withColumn(
    "rating",
    col("rating").cast("int")
    )

    df = df.filter(
        col("rating").between(1, 5)
    )

    result = df.groupBy("rating").count().orderBy(
            col("count").desc(),
            col("rating").asc()
        ).first()

    if result is not None:
        return result["rating"]
    return 0
```

Output:

```
5. Most Common Rating:
3
```

Driver code:

```python
# DRIVER CODE

spark = SparkSession.builder \
    .appName("Feedback Analysis") \
    .master("local[*]") \
    .getOrCreate()


# Create sample DataFrame
data = [
    ("alice@gmail.com", 5, "Excellent"),
    ("", 4, "Good"),
    (None, 3, "Average"),
    ("bob@gmail.com", 4, "Good"),
    ("charlie@gmail.com", 3, "Average"),
    ("david@gmail.com", 5, "Excellent")
]

columns = ["email", "rating", "comment"]

df = spark.createDataFrame(data, columns)


# 1. Load Feedback Data
# This function needs a CSV path, so normally the platform/tester calls it.
# For our manually created DataFrame, use df directly.
df.show()
```

```python
# 2. Clean Missing and Invalid Emails
cleaned_df = clean_missing_and_invalid_emails(df)

print("2. Cleaned Emails:")
cleaned_df.show()


# 3. Compute Rating Statistics
stats = compute_rating_stats(df)

print("3. Rating Statistics:")
print(stats)


# 4. Count Ratings
count = count_ratings(df)

print("4. Number of Ratings:")
print(count)


# 5. Most Common Rating
common_rating = most_common_rating(df)

print("5. Most Common Rating:")
print(common_rating)


spark.stop()
```